Testbericht 10 – CLI-Daemon

cad-cli (daemon, client, host, config), cad-ipc, cad-engine, cad-session, kernel-smlib · 23.10.2026 · Branch worktree-agent-aaefce8a64cd87ac5 · Commit 75d1f98 · rustc 1.97.0 (2d8144b78 2026-07-07) · 4 Kerne (Intel(R) Xeon(R) Processor @ 2.10GHz) · Messung mit `target/release/cad`

cargo test -- workspace	Daemon-Tests	clippy · fmt	render kalt → 2. Aufruf	Beschleunigung 2. Aufruf
353 / 353	**12 / 12**	**sauber · sauber**	**445–948 → 145–209 ms**	**2,8–25,8×**
0 ignoriert	crates/cad-cli/tests/daemon.rs	-D warnings	Median, drei Modelle	über alle 9 Messreihen

Leerer Aufruf (cube)	Speicherschätzung
32,7 / 30,4 ms	**1,01× (1,00–1,55)**
mit / ohne Daemon	Heap-Messung ÷ gehalten, 48 Modelle

1 Auftrag und Ergebnis

Bericht 8 maß für einen einzelnen `cad render` 0,4–1,1 s, davon der Kernel bis 640 ms; `cad watch` mit warmem Cache 24 ms. Jeder CLI-Aufruf begann mit leerem Cache. Jetzt hält ein Hintergrundprozess `cad daemon serve` den Engine-Cache über Aufrufe hinweg: der erste rechnende Befehl startet ihn, nach 15 min ohne Anfrage endet er, die Cache-Größe ist begrenzt, und mit `--no-daemon`, `CAD_NO_DAEMON=1` oder `[daemon] enabled = false` ist er aus. Ausgabe, Exit-Codes, Dateien und JSON sind dieselben wie ohne Daemon; ein zweiter Aufruf auf derselben Datei braucht 34–209 ms statt 287–1.357 ms.

2 Architektur

```
cad check|eval|inspect|render|export|outline|measure|set …     (Client, jeder Aufruf ein Prozess)
  1 cad-ipc: connect("daemon") → hello {proto, build_id, pid}
      kein Daemon  → cad daemon serve starten (setsid, stdin /dev/null, stdout/stderr → daemon.log),
                     bis 5 s auf hello warten
      andere Build-ID → stop (graceful, nach 3 s SIGKILL), neuen starten
      Fehler (kein Ordner, fremder Socket, Windows) → still lokal rechnen
  2 run {argv, cwd, stdin?} ──────────►  Verbindungs-Thread ── Kanal ──► Arbeits-Thread (256 MiB Stack)
     ◄── accepted, wait (1 s Takt)       (je Verbindung)                 eine Engine<SmlibKernel> für alle
Projekte
     ◄── stdout, stderr, exit, Dateien                                   chdir(cwd); Cli::try_parse_from(argv);
  3 Dateien schreiben (atomar bei set),                                  run(&cmd) mit Sitzung auf der Engine
    stdout/stderr ausgeben, Exit-Code                                    (host.rs: Dateien aufzeichnen, stdin)
                                      Überwachungs-Thread: Leerlauf 15 min, Watchdog 60 s, Socket vorhanden?
daemon.lock (flock, PID + Build) · daemon.sock (0600) · daemon.log  in  $CAD_IPC_DIR |
$XDG_RUNTIME_DIR/controller-cad | /tmp/controller-cad-<uid>
```

- **Gleicher Code mit und ohne Daemon.** Die Befehle der CLI laufen unverändert; drei Stellen gehen über `host.rs`: `load()` holt eine Sitzung über `host::new_session` (im Daemon auf dessen Engine, `Session::with_engine`; beim Ende des Befehls gibt `host::Pooled` die Engine zurück), stdin kommt vom Client, und Dateischreiben (`render`-PNG, `export`, `--all-parts`-Ordner, `set` atomar) wird im Daemon aufgezeichnet und vom Client ausgeführt. Ausgabe und Exit-Code baut dieselbe Funktion `finish()` im Daemon wie lokal. Schlägt das Schreiben beim Client fehl, rechnet er lokal neu und gibt so dieselbe Fehlermeldung.
- **Gleiche Flächen-Tags.** Die Engine vergibt Tags in der Reihenfolge, in der sie Knoteninhalte zum ersten Mal sieht; in einer Engine, die schon andere Modelle gerechnet hat, sind die Nummern andere als in einem frischen Prozess (sichtbar in `eval --json`: `nodes[].tag`, `tag_nodes`). Die Sitzung nummeriert Tags deshalb selbst in der Reihenfolge ihrer Jobs – genau die Nummern einer frischen Engine – und übersetzt Szene und Meshes (`render`

färbt über diese Zuordnung). Mit eigener Engine ist die Übersetzung die Identität und kostet nichts (Browser, Desktop).

- **Arbeitsverzeichnis:** der Daemon wechselt vor jedem Befehl in das Verzeichnis des Clients (danach nach `/`); relative Pfade, Meldungen und `include` verhalten sich wie lokal. Ein Arbeits-Thread, die Befehle laufen nacheinander (SMLib hat globalen Zustand).
- **Genau ein Daemon:** `flock` auf `daemon.lock` (neu in `cad-ipc`: `LockFile`, Datei beim Ende entfernt, Inode-Prüfung gegen das Entfernen durch den Vorgänger). Wer die Sperre nicht bekommt, wartet bis 10 s und endet, sobald ein Daemon antwortet; mehrere gleichzeitige Clients starten so höchstens Prozesse, die sofort wieder gehen.
- **Version:** `hello` liefert Protokoll und Build-ID (Version + Größe und Änderungszeit des Programms); bei Abweichung beendet der Client den alten Daemon und startet einen neuen.
- **Watchdog:** läuft ein Befehl länger als 60 s, meldet der Daemon den wartenden Clients „watchdog“ und endet (der hängende Kernel-Aufruf lässt sich nicht abbrechen, wie beim Engine-Prozess der Desktop-App). Der Client wiederholt den Befehl ohne Daemon und schreibt das auf stderr. Ohne Lebenszeichen 10 s (`wait` kommt jede Sekunde) beendet der Client den Daemon (SIGKILL) und rechnet selbst.
- **Sicherheit:** nur der Unix-Socket aus `cad-ipc` (Ordner 0700 des Benutzers, Socket 0600, Peer-UID), kein Netzwerk; Anfragen nur `hello`, `run`, `status`, `stop`; `run` nur für die acht Befehle (mit dem Parser der CLI geprüft). Der Daemon liest Dateien als derselbe Benutzer und schreibt keine.

Beispiel

```
$ cad check examples/spec/variabler_radius.scad --verbose
cad: Daemon gestartet (pid 20939, Log $TMP/ipc/daemon.log)
cad: Daemon pid 20939: 626 ms, Cache 1 Treffer / 11 neu, 11 Einträge, 15,4 MiB
examples/spec/variabler_radius.scad: 0 Fehler, 0 Warnungen
$ cad render examples/spec/variabler_radius.scad -o v.png --verbose
cad: Daemon pid 20939: 357 ms, Cache 12 Treffer / 1 neu, 12 Einträge, 15,6 MiB
$TMP/v.png: 1024x768, Ansicht iso, Hülle 160 × 30 × 20 mm
$ cad export examples/spec/lochplatte.scad -o l.step --verbose
cad: Daemon pid 20939: 925 ms, Cache 6 Treffer / 19 neu, 31 Einträge, 22,2 MiB
$TMP/l.step: 2 Körper, 23 Flächen, 49 Kanten, 28669 Bytes, Volumen 2376.6812 mm³
$ cad check examples/spec/lochplatte.scad --json
{"command": "check", "ok": true, "daemon": {"reason": null, "used": true}}
$ cad daemon status
Daemon läuft: pid 20939, Version 0.1.0 (Build 0.1.0+1e0f060.18e128aaa5a04ed5)
Laufzeit 2 s, Leerlauf 0 s (Ende nach 15 min ohne Anfrage), Watchdog 60 s
Anfragen: 4, Projekte 2 von 8 (0 verdrängt)
Cache: 27 Knoten + 4 Netze/abgeleitete, 22,2 MiB von 512,0 MiB, Trefferquote 58 % (42 Treffer, 31 neu
berechnet), 0 verdrängt
Speicher (RSS): 75,1 MiB, Start 35,3 MiB, Cache wird geleert über 803,3 MiB (0× geleert)
Socket $TMP/ipc/daemon.sock, Log $TMP/ipc/daemon.log
```

Log (`daemon.log` im Socket-Ordner):

```
[2026-10-23T12:37:03] cad daemon 20849: Ende (cad daemon stop)
[2026-10-23T12:37:05] cad daemon 20939: Start: Version 0.1.0, Build 0.1.0+1e0f060.18e128aaa5a04ed5, Cache 512
MiB / 16384 Einträge, 8 Projekte, Leerlauf 900 s, Watchdog 60 s, RSS 35,3 MiB
[2026-10-23T12:37:05] cad daemon 20939: `cad check examples/spec/variabler_radius.scad --verbose` → 0 in 626
ms, Cache 1 Treffer / 11 neu, 11 Einträge, 15,4 MiB
[2026-10-23T12:37:06] cad daemon 20939: `cad render examples/spec/variabler_radius.scad -o $TMP/v.png --
verbose` → 0 in 357 ms, Cache 12 Treffer / 1 neu, 12 Einträge, 15,6 MiB
[2026-10-23T12:37:07] cad daemon 20939: `cad export examples/spec/lochplatte.scad -o $TMP/l.step --verbose` → 0
in 925 ms, Cache 6 Treffer / 19 neu, 31 Einträge, 22,2 MiB
[2026-10-23T12:37:07] cad daemon 20939: `cad check examples/spec/lochplatte.scad --json` → 0 in 1 ms, Cache 23
Treffer / 0 neu, 31 Einträge, 22,2 MiB
```

3 Grenzen und Konfiguration

Einstellung	Standard	Datei `[daemon]`	Umgebung	`cad daemon serve`

Daemon an/aus	an	`enabled = false` (oder `daemon = false` oben)	`CAD_NO_DAEMON=1`	– (je Aufruf `--no-daemon`)
Cache (Knotenergebnisse + Meshes)	512 MiB	`cache_mb`	`CAD_DAEMON_CACHE_MB`	`--cache-mb`
Einträge	16384	`cache_entries`	`CAD_DAEMON_CACHE_ENTRIES`	–
Projekte im Cache	8	`max_projects`	`CAD_DAEMON_MAX_PROJECTS`	`--max-projects`
Leerlauf bis Ende	15 min	`idle_minutes`, `idle_secs`	`CAD_DAEMON_IDLE_MINUTES`, `_IDLE_SECS`	`--idle-minutes`, `--idle-secs`
Watchdog	60 s	`watchdog_secs`	`CAD_DAEMON_WATCHDOG_SECS`	`--watchdog-secs`
RSS-Grenze	Start + 1,5 × Cache	`rss_factor`	`CAD_DAEMON_RSS_FACTOR`	–

Datei: `$CAD_CONFIG`, sonst `$XDG_CONFIG_HOME/controller-cad/config.toml`, sonst `~/.config/controller-cad/config.toml`; Vorrang Argumente > Umgebung > Datei > Standard; ungültige Werte werden ignoriert (Log, mit `--verbose` auf stderr).

- **Cache:** ein LRU der Engine für Knotenergebnisse und Meshes zusammen (vorher: 4096 Knoten ohne Größe, Meshes 256 MiB getrennt), mit Bytegrenze und Eintragsgrenze; der neueste Eintrag bleibt auch, wenn er allein darüber liegt. Eine Engine für alle Projekte, die Schlüssel sind inhaltsbasiert.
- **Projekte:** je Hauptdatei (absoluter Pfad; stdin je Arbeitsverzeichnis) merkt sich der Daemon die Schlüssel, die ihre Befehle berührt haben. Beim 9. Projekt verliert das am längsten unbenutzte seine Einträge, soweit kein anderes gehaltenes Projekt sie nutzt.
- **RSS:** nach jeder Anfrage aus `/proc/self/statm`; über Start-RSS + 1,5 × Cache-Grenze wird der Cache geleert und freier Speicher mit `malloc_trim` zurückgegeben. Die Start-RSS (rund 35–40 MiB: Programm, SMLib, Tauri-Bibliotheken) zählt nicht gegen die Grenze, sonst griffe eine kleine Grenze immer.
- **Lebensdauer:** Ende nach 15 min ohne `run` (`status` zählt nicht), nach `cad daemon stop` (wartet auf laufende Befehle), wenn der Socket entfernt oder ersetzt wird (z. B. Testordner gelöscht), nach dem Watchdog; Socket und Sperrdatei werden entfernt.

Speicherschätzung

Kernel-Ergebnisse sind Handles in SMLib; ihre Größe kennt nur der C++-Heap. Gemessen mit `crates/cad-cli/examples/cache_size.rs`: jedes der 48 Beispielmodelle mit frischer Engine auswerten, dann die Engine fallen lassen und die freigegebenen Heap-Bytes zählen (glibc `mallinfo2`). Die Breps halten 10–297 KiB je Fläche (Median 26 KiB), Freiformflächen und variable Verrundungen am meisten. Die erste Schätzung aus der Topologie (4 KiB + 3 KiB/Fläche + 1 KiB/Kante) lag um den Faktor 6 zu niedrig. Jetzt: `Kernel::shape_bytes` für SMLib 8 KiB + 32 KiB je Fläche + 1 KiB je Kante (Summe über alle Beispiele 1,10×, je Modell 0,13–4,54×) und im Daemon zusätzlich eine Messung je Knoten: Heap nach minus vor der Auswertung des Knotens (`Engine::set_heap_probe`), was die neuen Breps des Knotens samt SMLib-Zwischendaten erfasst. Damit liegt die Schätzung je Modell bei 1,00–1,55× des tatsächlich Gehaltenen (Median 1,01), nie darunter.

Modell	Breps	Flächen	Kanten	gehalten (MiB)	Schätzung Topologie (÷ gehalten)	Daemon: Heap-Messung (÷ gehalten)
spec/freiformdeckel.scad	5	91	172	18,24	4,13 (0,23)	18,38 (1,01)
spec/variabler_radius.scad	11	169	336	14,91	5,71 (0,38)	15,25 (1,02)
spec/knauf_mirror.scad	16	48	67	13,92	1,81 (0,13)	16,39 (1,18)
spec/dose_shell.scad	6	218	404	12,13	10,80 (0,89)	12,48 (1,03)
spec/kabelfuehrung.scad	8	104	246	9,56	3,56 (0,37)	9,58 (1,00)
spec/loft_sweep.scad	6	45	87	8,78	1,54 (0,18)	8,79 (1,00)
openscad-compat/20_box_with_lid.scad	34	366	732	8,52	13,09 (1,54)	8,51 (1,00)

spec/griffschale.scad	16	201	462	6,45	8,53 (1,32)	6,96 (1,08)
alle 48 Modelle	**559**	**4989**	**9887**	**180,1**	**198,6 (1,10)**	**190,7 (1,06)**

Die acht größten Modelle; „gehalten“ = beim Verwerfen der Engine freigegeben. Meshes zählen mit 4 Byte je Array-Element (genau).

4 Messung

Je Zeile 5 Wiederholungen, jede mit gestopptem Daemon und frischer Kopie der Datei; Median (fett), daneben Minimum–Maximum, Wanduhrzeit des `cad`-Prozesses in ms (2026-10-23 12:37, 4 Kerne). „Kalt ohne Daemon“ = Verhalten bis Bericht 9. Kleine Änderungen:

- `spec/variabler_radius.scad`: Tasche `cube([30, 14, 10])` → `[30, 15, 10]` (ein Körper von dreien neu, mit variabler Verrundung)
- `projekt-gehaeuse/gehaeuse.scad`: Deckel um 1 mm verschoben (`tiefe + abstand + 1`; nur die Wurzel neu)
- `spec/lochplatte.scad`: Stift `d = 3.5` → `3.6` (Zylinder und Wurzel neu)

Modell	Befehl	kalt ohne Daemon	1. Aufruf mit Daemon (inkl. Start)	2. Aufruf	2. Aufruf nach kleiner Änderung	kalt ÷ 2.
spec/variabler_radius.scad	`check`	**625** 550–768	**695** 567–766	**36** 32–42	**162** 154–246	17,5×
spec/variabler_radius.scad	`render -o x.png`	**948** 890–1.163	**1.096** 978–1.326	**145** 139–205	**507** 493–619	6,5×
spec/variabler_radius.scad	`export -o x.step`	**1.357** 1.267–1.754	**1.313** 1.275–1.575	**63** 57–72	**1.003** 797–1.094	21,4×
projekt-gehaeuse/gehaeuse.scad	`check`	**287** 267–329	**331** 306–497	**35** 31–42	**36** 33–46	8,3×
projekt-gehaeuse/gehaeuse.scad	`render -o x.png`	**445** 427–465	**516** 457–662	**156** 149–161	**188** 183–229	2,8×
projekt-gehaeuse/gehaeuse.scad	`export -o x.step`	**409** 391–496	**425** 424–652	**34** 32–41	**153** 128–167	12,1×
spec/lochplatte.scad	`check`	**589** 519–759	**698** 586–882	**37** 32–43	**97** 73–154	15,7×
spec/lochplatte.scad	`render -o x.png`	**933** 874–1.090	**962** 851–1.031	**209** 190–224	**331** 327–392	4,5×
spec/lochplatte.scad	`export -o x.step`	**965** 811–1.338	**977** 858–1.530	**37** 34–44	**550** 403–666	25,8×

- Der 1. Aufruf mit Daemon kostet etwa so viel wie kalt ohne Daemon: im Median 44 ms mehr (Prozessstart des Daemons, Warten auf `hello`, eine zusätzliche Übertragung), je Messreihe -44 bis +149 ms – innerhalb der Streuung der Kaltzeit.
- Der 2. Aufruf rechnet nichts im Kernel: `check` braucht nur Parser, Auswertung und den Weg über den Socket; `render` zusätzlich Rasterisierung und PNG (das Mesh kommt aus dem Cache), `export -o x.step` den STEP-Writer.
- Nach einer kleinen Änderung rechnet der Kernel nur die betroffenen Knoten (Spalte „nach kleiner Änderung“). Bei `variabler_radius` dominiert die eine neue variable Verrundung; beim STEP-Export kommt die B-Rep der geänderten Wurzel dazu, die neu aus SMLib gelesen und geschrieben wird.
- Ein leerer Aufruf (`check` von `cube(1)`) kostet mit Daemon 32,7 ms, ohne 30,4 ms: Prozessstart des Clients und Socket statt SMLib-Initialisierung im Prozess.

5 Tests

- `cargo test --workspace`: 353 bestanden, 0 fehlgeschlagen, 0 ignoriert; `cargo clippy --workspace --all-targets -- -D warnings`: ohne Befund; `cargo fmt --all --check`: ohne Befund.

- Jeder Test hat einen eigenen Socket-Ordner (`CAD_IPC_DIR` in einem temporären Verzeichnis) und beendet seinen Daemon (Guard mit `cad daemon stop`); löscht der Test den Ordner, endet der Daemon spätestens 0,5 s später von selbst (Socket fehlt). Die übrigen CLI-Tests (`cli.rs`, `agent.rs`, `projects.rs`) laufen mit `CAD_NO_DAEMON=1`.
- CI: `scripts/ci/rust.sh test` prüft nach `cargo test`, dass kein `cad daemon serve` übrig ist (in CI, sonst Fehler); `scripts/ci/compat.sh compare|step` laufen jetzt über den Daemon (eigener Socket-Ordner, danach `status` und `stop`), so prüfen der OpenSCAD-Vergleich und die STEP-Prüfung mit OpenCASCADE auch die Ausgabe des Daemons; die Tauri-E2E-Tests rufen `cad` mit `CAD_NO_DAEMON=1` auf.

Test (`crates/cad-cli/tests/daemon.rs`)	Prüft	Ergebnis
`changed_files_are_never_stale`	Hauptdatei, eingebundene Datei, Rückkehr zu altem Stand, `set` über den Client, Syntaxfehler und Korrektur – jedes Ergebnis passt zum aktuellen Inhalt	ok
`autostart_then_cache_hits_and_faster`	erster Aufruf startet den Daemon (`daemon.used`), zweiter Aufruf: alle Knoten aus dem Cache, 0 Kernel-Aufrufe, gleiche Ausgabe, Trefferzähler steigt, messbar schneller	ok
`fallback_when_socket_is_broken`	Socket-Ordner ist eine Datei; fremder Prozess am Socket, der jede Verbindung schließt; veralteter Socket ohne Prozess (wird ersetzt)	ok
`idle_end_removes_socket_and_lock`	Leerlauf 1 s: Daemon endet, Socket und Sperrdatei entfernt, Log „ohne Anfrage“	ok
`parallel_starts_give_one_daemon`	sechs gleichzeitige Aufrufe ohne Daemon: alle nutzen den Daemon, genau ein Prozess `cad daemon serve` bleibt	ok
`relative_paths_and_stdin`	relativer Pfad mit anderem Arbeitsverzeichnis (gleiche Ausgabe), Quelltext über stdin (`-`)	ok
`status_and_stop`	`daemon status` (Text, JSON-Felder), `daemon stop` (PID, Socket entfernt), zweites `stop`: „kein Daemon“	ok
`switched_off_by_flag_env_and_config`	`--no-daemon` (vor und nach dem Befehl), `CAD_NO_DAEMON=1`, `[daemon] enabled = false`: kein Daemon, Grund in `daemon.reason`, Text still, `--verbose` nennt den Grund	ok
`version_change_replaces_daemon`	Build-ID a → b: Client beendet den alten Daemon und startet einen neuen; `status` eines dritten Builds meldet die Abweichung	ok
`watchdog_ends_daemon_and_command_runs_locally`	Watchdog 1 s, künstlich hängender Befehl (8 s): Daemon endet, Befehl läuft ohne Daemon mit gleicher Ausgabe, Meldung auf stderr, nächster Aufruf startet einen neuen Daemon	ok
`cache_limit_and_project_limit`	Grenze 1 MiB und 2 Projekte bei 7 Modellen: Verdrängungen, Größe unter der Grenze (oder nur der neueste Eintrag), 5 Projekte verdrängt; 512 MiB: nichts verdrängt	ok
`output_identical_with_and_without_daemon`	Daemon erst mit drei anderen Modellen gefüllt; dann je Befehl lokal, 1. und 2. Aufruf mit Daemon: check (ok, Syntaxfehler, Kernelfehler), eval, eval --module, inspect, outline, measure (Abstand/Volumen, Kollision/Abstand), set --dry-run, export mit unbekanntem Bauteil (Exit 2) – Text und JSON; render (Kontaktbogen, Schnitt, Röntgen) PNG byteweise, STL, 3MF, STEP (ohne Zeitstempel), --all-parts in einen Ordner	ok
`lock_file_is_exclusive_and_removed`	`cad-ipc`: Sperrdatei exklusiv (auch im selben Prozess), 0600, beim Ende entfernt	ok
`config::tests::file_values_and_errors`	Konfigurationsdatei: Werte, unbekannte Schlüssel, `daemon = false`, `enabled = false`, kaputtes TOML	ok
`cache::tests::evicts_least_recently_used`	LRU: Eintragsgrenze, `retain`, Verdrängungszähler	ok
`cache::tests::entry_limit_and_retain`	LRU: Eintragsgrenze, `retain`, Verdrängungszähler	ok

Im Test `autostart_then_cache_hits_and_faster` (Debug-Build, `eval --json` von `variabler_radius`): 1. Aufruf 680 ms, 2. Aufruf 49 ms.

6 Bekannte Grenzen

- Nur Unix (Linux, macOS; Peer-UID-Prüfung nur unter Linux, sonst schützt der 0700-Ordner). Unter Windows rechnet jeder Befehl selbst, still.
- Ein Arbeits-Thread: gleichzeitige CLI-Aufrufe warten aufeinander (ohne Daemon liefen sie parallel in eigenen Prozessen); `wait`-Ereignisse halten die Clients am Leben.
- Der Watchdog trifft auch einen legitim langen Befehl (> 60 s): der Daemon endet, der Befehl läuft ein zweites Mal ohne Daemon. Ein Absturz im Kernel (Signal) beendet den Daemon ebenso; die lokale Wiederholung kann genauso abstürzen. Getestet ist der Watchdog mit einer künstlichen Verzögerung (`CAD_DAEMON_DEBUG_HANG_MS`), nicht mit einem echten hängenden SMLib-Aufruf.
- Speichermessung je Knoten nur mit glibc; Allokationen anderer Threads (Verbindungen) zählen mit. Ohne glibc gilt die Topologie-Schätzung (je Modell Faktor 0,1–4,5). Die RSS wird nach jeder Anfrage geprüft, nicht während einer Auswertung.
- Abweichungen zur Ausgabe ohne Daemon: das zusätzliche JSON-Feld `daemon`; Zähler und Zeiten (`stats.cache_hits`, `kernel_calls`, `cached`, `*_ms`, `timings`, die Zeile „Statistik“ von `eval`) zeigen, was tatsächlich passiert ist. Bei einer Panik sind Meldung und Exit-Code gleich, die Panikmeldung des Laufzeitsystems steht aber im Log statt auf stderr.
- Build-ID aus Größe und Änderungszeit des Programms: wechseln sich zwei `cad`-Binaries ab (Debug und Release), ersetzen sie einander den Daemon (Cache weg). Ein alter Daemon, der noch rechnet, wird nach 3 s hart beendet; dessen Client rechnet dann selbst.
- Die Tag-Tabelle der Engine wächst weiter (ein Eintrag je gesehenem Knoteninhalt); nach 2^{20} Tags ersetzt der Daemon die Engine samt Cache.
- `cad watch` nutzt den Daemon nicht (eigener Cache im Prozess); `cad set` prüft die Änderung mit einer zweiten, kernel-losen Sitzung ohne Cache (wie bisher).

7 Reproduktion

```
export SMLIB_PREBUILT=$PWD/smlib/build-native
cargo build --release -p cad-cli --bin cad --example cache_size
python3 tools/report/report10-measure.py --cad target/release/cad --repeat 5
target/release/examples/cache_size examples/spec/*.scad examples/projekt-gehaeuse/gehaeuse.scad \
    examples/openscad-compat/*.scad > reports/tmp/report10/cache_size.jsonl
cargo test --workspace --no-fail-fast > reports/tmp/report10/cargo.txt 2>&1
cargo test -p cad-cli --test daemon -- --nocapture --test-threads 1 > reports/tmp/report10/daemon-tests.txt
2>&1
cargo clippy --workspace --all-targets -- -D warnings > reports/tmp/report10/clippy.txt 2>&1
cargo fmt --all --check > reports/tmp/report10/fmt.txt
node tools/report/report10.mjs
node tools/report/print-pdf.mjs docs/reports/10-cli-daemon/report.html docs/reports/10-cli-daemon/report.pdf
"Testbericht 10: CLI-Daemon"
```